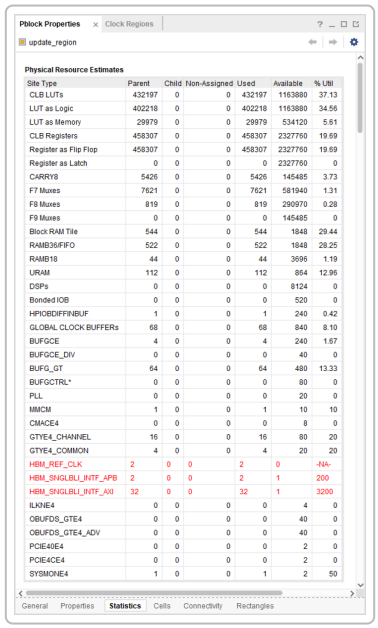
ChatGPT

Partial Reconfiguration (PR) and PR Regions

Partial reconfiguration (PR) divides an FPGA design into a **static region** and one or more **reconfigurable modules (RMs)** that can be swapped at run-time. The static design holds logic that never changes, while each RM (within a *PR region* or partition) can be independently implemented and reloaded. In Vivado's Dynamic Function eXchange (DFX) flow, each PR partition is defined by a *pblock* (floorplan) and an associated top-level module. After initial placement and routing, the logic belonging to each PR region is "carved out" and locked as static, with *partition pins* providing the interface to the RM [1]. Notably, Vivado allows the static logic to temporarily borrow unused resources in the PR region to improve routability [1]. Hierarchical PR (HPR) simply extends this concept: an RM itself can contain nested sub-partitions. In other words, **Nested DFX** lets one PR module be further subdivided into lower-level PR modules [2].

Defining PR Partitions and Floorplanning

In Vivado, one typically starts by **enabling the DFX flow** for the project, then marking modules as reconfigurable. This can be done via the DFX Wizard or Tcl commands, which create a "Partition Definition" for each PR region. Each partition gets a unique name and lists which source files or IP belong to that reconfigurable module. After the partitions are defined, the next step is **floorplanning** each region with a Pblock. A Pblock is a rectangular region in the FPGA fabric specified by commands (e.g. `resize_pblock`) in the XDC file or by dragging in the Vivado IDE. The Pblock reserves all the logic (CLB, DSP, BRAM, etc.) within it for that PR region. Vivado's floorplan editor provides detailed Pblock statistics: for each Pblock it lists the total available and used LUTs/FFs/DSPs/BRAMs, highlighting any resource shortfall (in red) that would require expanding the block [3] [4].



Site Type	Parent	Child	Non-Assigned	Used	Available	% Util
CLB LUTs	432197	0	0	432197	1163880	37.13
LUT as Logic	402218	0	0	402218	1163880	34.56
LUT as Memory	29979	0	0	29979	534120	5.61
CLB Registers	458307	0	0	458307	2327760	19.69
Register as Flip Flop	458307	0	0	458307	2327760	19.69
Register as Latch	0	0	0	0	2327760	0
CARRY8	5426	0	0	5426	145485	3.73
F7 Muxes	7621	0	0	7621	581940	1.31
F8 Muxes	819	0	0	819	290970	0.28
F9 Muxes	0	0	0	0	145485	0
Block RAM Tile	544	0	0	544	1848	29.44
RAMB36/FIFO	522	0	0	522	1848	28.25
RAMB18	44	0	0	44	3696	1.19
URAM	112	0	0	112	864	12.96
DSPs	0	0	0	0	8124	0
Bonded IOB	0	0	0	0	520	0
HPIOBDIFFINBUF	1	0	0	1	240	0.42
GLOBAL CLOCK BUFFERs	68	0	0	68	840	8.10
BUFGCE	4	0	0	4	240	1.67
BUFGCE_DIV	0	0	0	0	40	0
BUFG_GT	64	0	0	64	480	13.33
BUFGCTRL*	0	0	0	0	80	0
PLL	0	0	0	0	20	0
MMCM	1	0	0	1	10	10
GMCE4	0	0	0	0	8	0
GTYE4_CHANNEL	16	0	0	16	80	20
GTYE4_COMMON	4	0	0	4	20	20
HBM_REF_CLK	2	0	0	2	0	-NA-
HBM_SNGLBLI_INTF_APB	2	0	0	2	1	200
HBM_SNGLBLI_INTF_AXI	32	0	0	32	1	3200
ILKNE4	0	0	0	0	4	0
OBUFDS_GTE4	0	0	0	0	40	0
OBUFDS_GTE4_ADV	0	0	0	0	40	0
PCIE40E4	0	0	0	0	2	0
PCIE4CE4	0	0	0	0	2	0
SYSMONE4	1	0	0	1	2	50

In Vivado's floorplanning view, the **Pblock Properties** panel shows resource counts for each partition. For example, it reports how many LUTs or BRAMs are available versus used by the current reconfigurable module. This lets the designer verify that the Pblock is large enough – if any category is over-utilized (shown in red), the Pblock must be enlarged [3] [4]. In practice, one may iteratively adjust Pblocks so that each PR region has just enough resources (with a safety margin) for its largest module.

Once partitions and Pblocks are defined, Vivado can infer **partition pins** automatically at the PR boundaries, or the designer can add explicit black-box ports. These pins form the static/RM interface and carry data, clocks, resets, etc. Proper floorplanning (e.g. aligning Pblocks to columns, minimizing fragmentation) is critical to ensure routability and to avoid mixing static and dynamic nets in the same frame. In general, after partitioning is set, a first place-and-route is run to create the initial design context.

Implementation Flow (Fixed PR)

In a fixed-PR (flat) design, after one full implementation of the *static design with one RM*, Vivado produces a set of checkpoint files. In particular, it generates:

- **Full routed checkpoint** (e.g. `top_routed.dcp`) containing the complete design with one instance of each RM placed and routed.
- **Static-only checkpoint** (e.g. `top_routed_bb.dcp`), where each PR partition is black-boxed and its routing is locked (so the static part is frozen).
- **Module-level routed checkpoints** for each RM instance (e.g. `u_count_count_up_routed.dcp`), which contain just that RM's logic placed and routed within the context of the parent design.

This is automatic in Vivado's PR flow [5]. The static-only checkpoint (the *abstract shell*) contains only the nets and placement relevant to static logic, with empty black-boxes for the RMs. In practice, this static-only DCP is much smaller than the full design: for example, one demo had a 58 MB full DCP and a 56 MB static-only DCP, whereas each RM's checkpoint was only a few MB [5]. These checkpoints are then used for incremental implementation of each RM.

The typical fixed-PR compile steps are:

1. **Synthesize** the top-level and each RM (often out-of-context for each RM).
2. **Run initial Implementation** (place & route) for the parent configuration (one RM in each partition). Vivado will produce the checkpoints above automatically [5].
3. **Generate Abstract Shell (Static-only DCP)**. This is done by black-boxing all RMs in the parent checkpoint and locking placement (`update_design -black_box`, `lock_design`) or by using Vivado's DFX flow which does it internally [5] [6].
4. **Implement each RM separately**: for each partition and target RM, load the abstract-shell DCP into Vivado and P&R only that RM's logic (other partitions remain black-boxed). This uses much less memory and can be parallelized [6] [7].
5. **Generate Partial Bitstreams** for each RM. Vivado writes a partial BIT file for each RM/partition.

After these steps, you have one partial bitstream per RM configuration (plus optionally a full configuration bitstream if needed). The *static bitstream* (full design) is loaded first, then later any RM can be reconfigured by loading its partial bitstream over the same frames.

Abstract Shell Flow

To speed up RM compilation, Vivado supports an **abstract-shell mode**. Instead of repeatedly loading the full static design (which can be very large), Vivado uses a minimal "shell" context. The abstract shell is a checkpoint that contains only the static logic and placement (with fixed routing), and no detailed routing for the RMs [6]. It includes just enough physical context so that an RM can be placed and routed for timing, but not the entire static router database. This approach has several benefits: it *reduces compile time and memory* for each RM implementation, *reduces the size* of the static checkpoint on disk, and it enables **parallel compilation** of multiple RMs, since the full static context does not need to be loaded for each run [6] [7]. It also helps with IP protection: proprietary static logic can be hidden (compiled into the abstract shell) so that only black boxes are exposed to the RM compile steps [7]. In practice, Vivado's DFX flow creates abstract-shell checkpoints automatically after the parent P&R run [5], so no manual steps are needed beyond using the generated checkpoints.

Hierarchical PR (Nested DFX)

Hierarchical partial reconfiguration (HPR) takes the above flow further by allowing *nested* reconfigurable partitions. In Xilinx Vivado (2023+), this is called *Nested DFX*. The idea is that you can take an existing PR region and subdivide it into smaller sub-regions, each of which can then host its own RM. This supports more granular reconfiguration without re-routing the entire device.

The HPR process is as follows [8] [9]: after completing the initial (top-level) implementation and generating the abstract shell, you run the Tcl command `pr_subdivide`. This takes a target parent partition (cell) and a list of child partitions, and it *breaks the parent RP into new lower-level RPs*. Vivado's `pr_subdivide` automatically black-boxes the parent RP and loads a synthesis checkpoint for the parent logic. It then "pushes down" the reconfigurable property into the new child cells, effectively creating new partitions within the old one [9]. The original parent partition is marked as a `HD.RECONFIGURABLE_CONTAINER` (a placeholder), and the lower-level partitions get the normal `HD.RECONFIGURABLE` property.

Once the sub-partitions are defined, you floorplan each (usually by splitting the original Pblock area) and implement each new RM in context of the locked static region (just as before, but now the static region includes the placed parent RM logic). After generating partial bitstreams for all child RMs, you then use the command `pr_recombine` on the parent partition [10]. This removes the lower-level RPs and *re-establishes the parent cell* as a reconfigurable partition by moving the `HD.RECONFIGURABLE` property back to it and removing it from its children [10]. The result is that the parent partition is restored (but the static logic now includes the static results of all child implementations). Finally, you can generate the parent-level bitstream in the usual way.

Importantly, at run-time one must load the bitstreams in order: **first the upper-level (parent) bitstream**, then the sub-partitions. In other words, you load the full context setting from the parent region before loading any of the smaller ones [8]. This hierarchical sequence sets up the correct context for each level.

Comparison: Fixed-Size vs Hierarchical PR

Fixed-size PR (flat partitions) is simpler to manage but can be less flexible. In a fixed approach, the designer defines one set of PRR sizes up front. If a given module is smaller than its partition, area is wasted (internal

fragmentation); if larger, the design must be reshaped or cannot fit. Hierarchical PR (with variable-sized "pages") allows resizing partitions post hoc. For example, one research overlay partitioned the fabric into a grid of small "PR pages" that could be combined (two adjacent pages into one double-size page, etc.) to accommodate larger modules. The result was that each operator could be compiled on a just-right-size region, improving compile speed and utilization [11] [12]. Indeed, experiments on an UltraScale+ device (ZU9EG) showed that variable-sized hierarchical PR achieved 1.4–4.9× higher performance than fixed-page PR, while compilation ran 2.2–5.3× faster [12]. This is because smaller regions compile faster and in parallel, yet large modules can still be mapped by merging regions on the fly.

However, HPR adds complexity. There is overhead in carving and re-merging partitions, and the static design ends up containing more submodules (which may block routing). In practice, Xilinx's Nested DFX commands automate much of this, but the designer must still manage multiple levels and loading sequences. In summary: **fixed PR** is straightforward but may under-utilize fabric if partition sizes are suboptimal. **Hierarchical PR** is more flexible and can significantly shorten compile time (by separate compilation) and improve performance, at the cost of extra design-flow steps and slightly more complex bitstream management [12] [8].

Routability, Fragmentation, and Blocked Resources

Efficient PR design requires careful handling of routing and resource fragmentation. For routability, it is important to allow the static logic to use any free wires in the PR regions: Vivado does this automatically by borrowing routes in a Pblock for static nets if those wires are otherwise unused. Additionally, one can improve routability by **tight packing**: instantiate each RM so it nearly fills its Pblock. Doing so pushes static logic out of the PR area, reducing the amount of static routing that *bleeds into* the PR region. In practice, tight RMs result in more clearly separated static vs dynamic routing, which minimizes the number of *blocked* routing resources (those reserved for decoupling at PR boundaries) [13]. (By contrast, artificially placing a "blocker" net at the boundary can also reduce blocked sites, but this makes routing and timing more difficult.) In hierarchical flows, each subdivision tends to increase blocked resources due to additional netlist hierarchy and decoupling, so the packing strategy becomes even more important [14] [13].

Vivado actually reports which resources are blocked around each partition (e.g. in `blockedBelsOutputs.tcl`). A sound flow will **exclude** these blocked resources when estimating a PR area budget [15]. For example, one framework parsed Vivado's blocked-site reports and subtracted those LUTs/BRAMs from each PR page's capacity before assigning modules [15]. Designers also typically include a safety margin on resource usage: it is common to reserve about 10–20% of each Pblock as slack to avoid congestion. That is, an RM whose utilization times (1+margin) exceeds the block's capacity is considered too large [16]. In practice, a margin of 10–15% is often used, raised to ~20% in very tight regions, to account for routing blow-up.

Fragmentation (wasted space) is addressed by smart placement of RMs. Internally, a PR module smaller than its block leaves unused area inside the block (internal fragmentation). Externally, scattered small modules can leave other blocks underutilized (external fragmentation). One effective technique is a **greedy page-assignment**: sort RMs by size (e.g. sum of LUT%, BRAM%, DSP% of the chip), then place the largest first. Each RM is fitted into the smallest available block (or combination of blocks) that will hold it [17]. This minimizes internal fragmentation by not over-sizing blocks, and minimizes external fragmentation by ensuring big modules aren't split into too-small pieces [17]. If an operator doesn't fit any single block, one

can try merging blocks (double or quad pages) in the layout and re-running implementation, or move to the next level of HPR.

In iterative development, the greedy allocator can reuse previous placements. After changes to the design (new versions of some RMs), the tool first checks if the unchanged modules still fit in their old assigned pages. If so, those need not be recompiled – only the changed modules are re-synthesized and placed. This incremental strategy saves minutes of compile time by avoiding full recompilation when only a few blocks changed [18]. In one experiment with ~22 operators, this page-assignment and reuse logic completed in under a second and ensured that only modified RMs were rebuilt [18].

Incremental Compilation and Productivity

Modern Vivado flows support project-based incremental compilation, where intermediate results (placed netlists, routing) are preserved between runs. Combined with abstract shells, this means that once the static region is implemented, adding or changing a single RM requires only a quick place-and-route of that RM. The remainder of the design is simply reloaded from previous data. As noted above, a greedy assignment algorithm can detect unchanged RMs and skip them. The net effect is that designers can rapidly iterate: one study reported compiling all operators in an overlay in 2–5 minutes using nested PR (versus 7–22 minutes in a monolithic Vitis compile) [12]. By contrast, a fixed-size PR approach might force rebuilding large islands whenever any part changes.

Case Study: Versal ZU9EG (ZCU102) Platform

These techniques have been demonstrated on Xilinx's high-end FPGA platforms. For instance, researchers built a custom DFX "overlay" on the ZCU102 board (which uses an UltraScale+ ZU9EG device) with a pre-built network-on-chip and dozens of PR pages [19]. They expanded the official DFX dynamic region to use ~264K LUTs, ~1.1M FFs, 72 MB RAM, and 2,448 DSPs of the chip [19]. Within this dynamic region they defined 22 "pages" (each ~7K–18K LUTs) and showed how these could be incrementally merged into 11 double-pages or 5 quad-pages to fit larger operators [20] [19]. Using nested DFX on this Versal platform, they achieved very fast reconfiguration times and high throughput for dataflow kernels, illustrating the power of hierarchical PR on modern hardware [19] [12].

Sources: Xilinx/AMD Vivado documentation (UG909) on Dynamic Function eXchange and Nested DFX [2] [1] [5] [6] [9] [10]; 2022–2024 FPGA research (e.g. FPT 2022) on hierarchical PR overlays and page-based scheduling [12] [18] [20] [15] [13].

---

1 2 8 11 12 13 14 15 16 17 18 19 20 ic.ese.upenn.edu
https://ic.ese.upenn.edu/pdf/nested_dfx_fpt2022.pdf

3 4 Manage the Reconfigurable Partition Floorplan - 2024.2 English - UG909
https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/Manage-the-Reconfigurable-Partition-Floorplan

5 Step 3: Create Abstract Shells - 2024.2 English - UG947
https://docs.amd.com/r/en-US/ug947-vivado-partial-reconfiguration-tutorial/Step-3-Create-Abstract-Shells

6 7 Abstract Shell for Dynamic Function eXchange - 2024.2 English - UG909
https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/Abstract-Shell-for-Dynamic-Function-eXchange

9 pr_subdivide - 2024.2 English - UG909
https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/pr_subdivide?contentId=gPVpmhDxdMWGMiANiiS_gQ

10 pr_recombine - 2024.2 English - UG909
https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration/pr_recombine?contentId=x3G2KGX5uqtn1pCswWPysw